

myMove Development Roadmap

Phase 1: Foundation Setup (Week 1-2)

Week 1: Environment Setup

- ☐ Install Flutter SDK (latest stable version)
- ☐ Set up Android Studio / VS Code with Flutter extensions
- ☐ Create Firebase project
- ☐ Configure Firebase for Android
- ☐ Set up version control (Git)
- ☐ Create project structure following best practices

Week 2: Firebase Configuration

- ☐ Enable Firebase Authentication (Email/Password, Google Sign-In)
- ☐ Set up Cloud Firestore database
- ☐ Set up Realtime Database for ESP32 integration
- ☐ Configure Firebase Cloud Messaging (FCM)
- ☐ Set up Firebase Storage
- ☐ Deploy Firebase Security Rules
- ☐ Create initial data models in Firestore

Phase 2: Core Authentication (Week 3)

User Authentication

- ☐ Implement login screen UI
- ☐ Implement registration screen UI
- ☐ Create AuthProvider with state management
- ☐ Implement email/password authentication
- ☐ Implement Google Sign-In
- ☐ Add phone number verification (optional)
- ☐ Create user profile in Firestore on registration
- ☐ Implement password reset functionality
- ☐ Add form validation
- ☐ Test authentication flow

Phase 3: Hardware Integration (Week 4-5)

ESP32 Setup

- ☐ Purchase ESP32 boards and ultrasonic sensors (HC-SR04)

- ☐ Set up Arduino IDE for ESP32
- ☐ Install required libraries (Firebase ESP Client)
- ☐ Upload sensor code to ESP32
- ☐ Test WiFi connectivity
- ☐ Test Firebase Realtime Database connection
- ☐ Calibrate distance detection threshold
- ☐ Set up multiple ESP32s for different parking spots
- ☐ Create database structure for parking spots

Testing Hardware

- ☐ Test real-time data updates
- ☐ Verify occupied/vacant detection accuracy
- ☐ Test connection stability
- ☐ Monitor battery/power consumption
- ☐ Document hardware setup process

Phase 4: Parking Search & Discovery (Week 6-7)

Home Screen

- ☐ Implement Google Maps integration
- ☐ Add location permission handling
- ☐ Display user's current location
- ☐ Show nearby parking buildings as markers
- ☐ Create custom map markers for buildings
- ☐ Add search functionality
- ☐ Implement filter options (price, distance, availability)

Building Details

- ☐ Create building details screen
- ☐ Display building information (name, address, hours)
- ☐ Show pricing information
- ☐ Display total/available spots count
- ☐ Add directions/navigation button
- ☐ Implement real-time availability updates

Phase 5: Parking Spot Selection (Week 8)

Floor & Spot Selection

- ☐ Create parking spot grid UI
- ☐ Implement floor selector

- ☐ Stream real-time spot availability from Firebase
- ☐ Color-code spots (available/occupied/reserved)
- ☐ Add spot selection functionality
- ☐ Display spot details
- ☐ Prevent selection of occupied spots
- ☐ Update UI when spots change in real-time

Phase 6: Booking System (Week 9-10)

Booking Flow

- ☐ Create booking form screen
- ☐ Add date/time picker
- ☐ Calculate parking duration
- ☐ Calculate parking fees
- ☐ Implement spot reservation in Firebase
- ☐ Add vehicle selection
- ☐ Create booking review screen
- ☐ Implement booking creation
- ☐ Generate booking confirmation

Active Booking Management

- ☐ Create active booking screen
- ☐ Display parking timer/countdown
- ☐ Show remaining time
- ☐ Add extend parking functionality
- ☐ Implement early checkout
- ☐ Send parking expiry reminders
- ☐ Update booking status in real-time

Phase 7: Payment Integration (Week 11)

Stripe Setup

- ☐ Create Stripe account
- ☐ Install Flutter Stripe package
- ☐ Configure Stripe API keys in Firebase
- ☐ Implement payment method selection
- ☐ Create payment intent via Cloud Function
- ☐ Handle payment confirmation
- ☐ Process successful payments
- ☐ Handle payment failures

- ☐ Store payment records
- ☐ Generate receipts

Alternative Payment Methods

- ☐ Research Touch 'n Go eWallet API
- ☐ Implement alternative payment gateways
- ☐ Add multiple payment options

Phase 8: QR Code System (Week 12)

QR Generation

- ☐ Generate unique QR codes for bookings
- ☐ Create QR display screen
- ☐ Add QR code to booking confirmation
- ☐ Store QR data in Firebase

QR Scanning

- ☐ Implement QR scanner screen
- ☐ Add camera permissions
- ☐ Parse scanned QR data
- ☐ Validate QR code authenticity
- ☐ Extract user information
- ☐ Create contact options dialog

Phase 9: Communication Features (Week 13-14)

In-App Messaging

- ☐ Set up Firestore for messages
- ☐ Create chat screen UI
- ☐ Implement send/receive messages
- ☐ Add message timestamps
- ☐ Display message history
- ☐ Send message notifications
- ☐ Mark messages as read
- ☐ Add privacy controls

VoIP Calling (using Agora)

- ☐ Set up Agora account
- ☐ Integrate Agora SDK
- ☐ Implement call initiation

- ☐ Create call screen UI
- ☐ Handle incoming calls
- ☐ Add call controls (mute, speaker, end)
- ☐ Test call quality
- ☐ Handle call notifications

Phase 10: Notifications (Week 15)

Push Notifications

- ☐ Configure FCM in Flutter
- ☐ Request notification permissions
- ☐ Save FCM tokens to Firestore
- ☐ Implement notification handlers
- ☐ Create Cloud Functions for notifications

Notification Types

- ☐ Booking confirmation
- ☐ Parking expiry warning (15 min before)
- ☐ Someone scanned your QR code
- ☐ Payment successful/failed
- ☐ Reservation auto-released
- ☐ Custom alerts

Phase 11: Additional Features (Week 16-17)

Navigation

- ☐ Integrate Google Maps Directions API
- ☐ Implement turn-by-turn navigation
- ☐ Show estimated arrival time
- ☐ Add traffic information

Booking History

- ☐ Create booking history screen
- ☐ Display past bookings
- ☐ Show booking details
- ☐ Add download receipt option
- ☐ Implement search/filter

User Profile

- ☐ Create profile screen

- ☐ Add edit profile functionality
- ☐ Manage vehicles
- ☐ Add/remove payment methods
- ☐ View parking statistics
- ☐ Settings and preferences

Reviews & Ratings

- ☐ Add rating system for buildings
- ☐ Implement review submission
- ☐ Display average ratings
- ☐ Show user reviews

Phase 12: Testing & Optimization (Week 18-19)

Functional Testing

- ☐ Test authentication flows
- ☐ Test booking creation
- ☐ Test payment processing
- ☐ Test real-time updates
- ☐ Test QR scanning
- ☐ Test communication features
- ☐ Test notifications

Performance Testing

- ☐ Test app load times
- ☐ Optimize image loading
- ☐ Test with poor internet connection
- ☐ Check memory usage
- ☐ Test battery consumption
- ☐ Optimize database queries

User Testing

- ☐ Conduct usability testing (10+ users)
- ☐ Collect feedback
- ☐ Identify pain points
- ☐ Make UI/UX improvements
- ☐ Fix reported bugs

Phase 13: Deployment Preparation (Week 20)

App Store Preparation

- ☐ Create app icons
- ☐ Design splash screen
- ☐ Write app description
- ☐ Create screenshots
- ☐ Record demo video
- ☐ Prepare privacy policy
- ☐ Prepare terms of service

Final Checks

- ☐ Code review
- ☐ Remove debug code
- ☐ Update app version
- ☐ Generate release APK
- ☐ Test release build
- ☐ Prepare deployment documentation

Phase 14: Deployment & Launch (Week 21)

Deployment

- ☐ Deploy Cloud Functions
- ☐ Update Firebase configuration
- ☐ Set up Firebase hosting (for web dashboard)
- ☐ Submit to Google Play Store
- ☐ Monitor crash reports
- ☐ Set up analytics

Post-Launch

- ☐ Monitor user feedback
- ☐ Track app metrics
- ☐ Fix critical bugs
- ☐ Plan future updates

Future Enhancements (Post FYP2)

Advanced Features

- ☐ AI-powered parking predictions
- ☐ Multi-building parking subscriptions

- ☐ EV charging spot integration
- ☐ Valet parking service
- ☐ Corporate parking management
- ☐ Integration with city council systems
- ☐ iOS app development
- ☐ Web dashboard for building managers

Hardware Upgrades

- ☐ License plate recognition cameras
- ☐ Automatic barrier gates
- ☐ Mobile app-controlled barriers
- ☐ Solar-powered sensors
- ☐ Advanced analytics dashboard

Key Milestones

1. **End of Week 5:** Hardware working, basic app skeleton ready
2. **End of Week 10:** Core booking system functional
3. **End of Week 14:** All major features implemented
4. **End of Week 19:** Testing complete, ready for deployment
5. **Week 21:** App launched on Play Store

Resources Needed

Hardware

- 10-20 ESP32 boards (~RM 30-50 each)
- 10-20 HC-SR04 ultrasonic sensors (~RM 5-10 each)
- Power supplies/adapters
- WiFi router for testing
- Test environment (building with parking spaces)

Software/Services

- Firebase Spark Plan (free) → Blaze Plan (pay-as-you-go)
- Stripe account (no setup fee, transaction fees apply)
- Agora free tier (10,000 minutes/month)
- Google Cloud Console (for Maps API)
- Domain name (optional, ~RM 50/year)

Estimated Costs

- Hardware: RM 500 - 1,000
- Firebase (monthly): RM 50 - 200 (depending on usage)
- Agora: Free (within limits)
- Stripe: Per-transaction fees ($\sim 2.9\% + \text{RM } 0.50$)
- Total initial investment: $\sim \text{RM } 2,000 - 3,000$

Tips for Success

1. **Start Simple:** Build MVP first, add features incrementally
2. **Test Early:** Test hardware integration from day one
3. **Version Control:** Commit code regularly with meaningful messages
4. **Documentation:** Document your code and APIs
5. **Seek Help:** Use Flutter community, Stack Overflow when stuck
6. **User Feedback:** Test with real users throughout development
7. **Stay Organized:** Use project management tools (Trello, Notion)
8. **Monitor Progress:** Track against Gantt chart weekly